

Работа 1: Разработка защищенного REST API с интеграцией в CI/CD

Назначение: получить практический опыт разработки безопасного backend-приложения с автоматизированной проверкой кода на уязвимости. Освоить принципы защиты от OWASP Top 10 и интеграцию инструментов безопасности в процесс разработки.

Задание

Выполните следующие шаги для создания простого, но защищенного веб-API:

1. Выбор стека и инициализация проекта:

- Выберите язык программирования (Python/Flask, JavaScript/Node.js + Express, Java).
- Создайте новый проект с помощью менеджера пакетов (npm, pip, maven).
- Инициализируйте git-репозиторий и свяжите его с удаленным репозиторием на GitHub или GitLab.

2. Разработка функционального API:

- Реализуйте 3 метода:
 - 1) POST /auth/login — метод для аутентификации пользователя (принимает логин и пароль).
 - 2) GET /api/data — метод для получения каких-либо данных (например, список пользователей или постов). Доступ должен быть только у аутентифицированных пользователей.
 - 3) Придумайте сами
- Для хранения данных используйте любую удобную базу данных (SQLite для простоты или PostgreSQL) или даже простой файл/массив в памяти, если это учебный пример.

3. Внедрение базовых мер защиты:

- **Защита от SQLi (SQL-инъекций):** Используйте параметризованные запросы (Prepared Statements) или ORM (например, Sequelize, SQLAlchemy, Hibernate). Не используйте конкатенацию строк для формирования SQL-запросов.
- **Защита от XSS:** Санитизируйте (экранируйте) все пользовательские данные, которые возвращаются в ответах API. Используйте встроенные функции фреймворка (например, escape() в Express).
- **Защита от Broken Authentication:**
 - Реализуйте выдачу JWT-токена upon successful login.
 - Напишите middleware, которое будет проверять JWT-токен на всех защищенных эндпоинтах.

- Пароли ни в коем случае не должны храниться в открытом виде. Обязательно хэшируйте их с помощью алгоритмов типа bcrypt, scrypt или Argon2.

4. Настройка CI/CD pipeline с security-сканерами:

- В корне проекта создайте конфигурационный файл для GitHub Actions (.github/workflows/ci.yml) или GitLab CI (.gitlab-ci.yml).
- **SAST (Static Application Security Testing):** Настройте в pipeline запуск статического анализатора кода.
 - Для Python: bandit или safety
 - Для JavaScript: npm audit или snyk test
 - Для Java: spotbugs или использование плагина OWASP Dependency-Check
- **SCA (Software Composition Analysis):** Настройте проверку зависимостей на известные уязвимости.
 - Используйте OWASP Dependency-Check или Snyk.
 - Настройте step в pipeline, который будет запускать сканер и выдавать отчет.
- При каждом push в репозиторий или создании pull request должны автоматически запускаться проверки.

5. Тестирование и документирование:

- Протестируйте работу API с помощью Postman или curl. Убедитесь, что аутентификация работает, а доступ без токена — запрещен.
- Сделайте скриншоты отчетов от SAST/SCA-инструментов, которые показывают, что проверки прошли успешно (не найдено критических уязвимостей) или что найденные уязвимости были вами исправлены.

Что сдают студенты

1. Ссылка на **публичный git-репозиторий** (GitHub/GitLab) с кодом проекта.
2. Текст файла README.md из репозитория, содержащий:
 - Описание проекта и API (какие эндпоинты есть, как их вызывать).
 - **Подробное описание реализованных мер защиты** (как именно защищены от SQLi, XSS, как реализована аутентификация).
 - Скриншоты отчетов SAST/SCA из раздела "Actions" / "CI/CD" вашего репозитория.
3. Ссылка на последний успешный запуск pipeline в вашем репозитории.

Критерии оценки (макс. 10 баллов)

- **3 балла:** Работоспособность API. Эндпоинты аутентификации и получения данных возвращают корректные ответы.

- **4 балла:** Реализация мер защиты:
 - 1 балл за защиту от SQLi.
 - 1 балл за защиту от XSS.
 - 2 балла за корректную реализацию аутентификации (JWT + хэширование паролей).
- **2 балла:** Настройка CI/CD pipeline с запуском как минимум одного SAST/SCA инструмента.
- **1 балл:** Качество оформления отчета (четкость, структурированность, наличие скриншотов), качество документации и оформления кода.

Источники для теории

1. **OWASP Top 10:** <https://owasp.org/www-project-top-ten/> (Обратите внимание на A01:2021-Broken Access Control, A03:2021-Injection, A07:2021-Identification and Authentication Failures).
2. **OWASP Cheat Sheet Series:** <https://cheatsheetseries.owasp.org/> (В частности, cheat sheets по Authentication, SQL Injection Prevention, XSS Prevention).
3. **Документация по JWT (JSON Web Tokens):** <https://jwt.io/introduction>
4. **Документация по GitHub Actions:** <https://docs.github.com/en/actions> или **GitLab CI:** <https://docs.gitlab.com/ee/ci/>

Контрольные вопросы

1. Почему хэширование пароля с помощью алгоритмов вроде bcrypt предпочтительнее использования SHA-256?
2. В чем заключается основная разница между SAST и DAST-подходом к тестированию безопасности?
3. Опишите механизм работы JWT-токена. Какая информация в нем содержится и как сервер проверяет его подлинность?
4. Какие риски возникают, если не проводить аудит сторонних библиотек (зависимостей) в проекте?